Lab 3 Report: A LiDAR-Enabled Approach To Autonomous Wall Following and Collision Aversion

Team 2

Brendon Murphy
Nina Ruiz-Garcia
Irene Tang
Vandell Vatel

RSS

March 13, 2026

Editor: Vandell Vatel

1 Introduction

Author: Vandell Vatel

Autonomous vehicles utilize programs such as a wall follower and a safety controller to help them safely and accurately navigate the world. The wall follower acts as a guidance system in situations where we must maintain a desired distance from a boundary, while the safety controller is a feature that helps us prioritize the safety of passengers and bystanders. Through this lab, we were tasked with the development of a safety controller that would supersede our previously written wall follower when necessary to prevent our race-car from crashing into obstacles. The technicality lies in tuning our safety controller so that it strikes a balance of risk-aversion and risk-tolerance. If the controller is too risk-averse, it will cause our race-car to stop when it is not in danger of crashing, causing it to perform sub-optimally in future labs. However, if it is too risk-tolerant, it either won't stop or won't stop early enough to prevent a crash. Our solution is a safety controller that constantly calculates a projected path that our race-car would take in a short time frame. If an obstacle is detected within a designated safety radius, meaning that we would collide with the object within the time frame, the controller takes priority and forces the race-car to stop.

2 Technical Approach

This section summarizes the control architecture used for two key vehicle functions: wall following and safety control.

Wall Following

Author: Irene Tang

The goal of the wall follower is to maintain a set distance away from a wall while driving beside it. The input signal for the car is the necessary steering angle to maintain the appropriate distance. To achieve this, the wall-following pipeline does two things:

- Wall Estimator: fits a line to lidar measurements corresponding to the nearby wall.
- Path Controller: computes steering commands to maintain a desired offset from the wall while aligning the vehicle with it.

1. Wall Estimator

The estimator constructs a linear approximation of the wall using a subset of lidar scan points. Only scan points likely to belong to the side wall are retained. We restrict measurements to an angular window $[-\pi/12, \pi/2]$ if following the left wall or $[-\pi/12, \pi/2]$ if following the right wall and distance range $[0.2, 5]$ m to remove irrelevant returns. The angular range includes angles in front of the car to estimate corners and handle turning behavior. A straight line is then fit to the filtered points $\{(x_i, y_i)\}$ using ordinary least squares:

$$\theta_{left} \in [-\pi/12, \pi/2]$$
$$\theta_{right} \in [-\pi/2, \pi/12]$$
$$d \in [0.2\text{m}, 5\text{m}]$$
$$(m, b) = \arg\min_{m,b} \sum_i (y_i - mx_i - b)^2 \tag{1}$$

This line provides two quantities used by the controller:

- The perpendicular distance from the vehicle to the wall

$$d^{\text{wall}}_{\text{estimated}} = \frac{-b}{\sqrt{m^2 + 1}} \tag{2}$$

- The wall orientation relative to the vehicle

$$\theta_{\text{wall}} = \arctan(m) \tag{3}$$

These values form the wall state estimate used for feedback control.

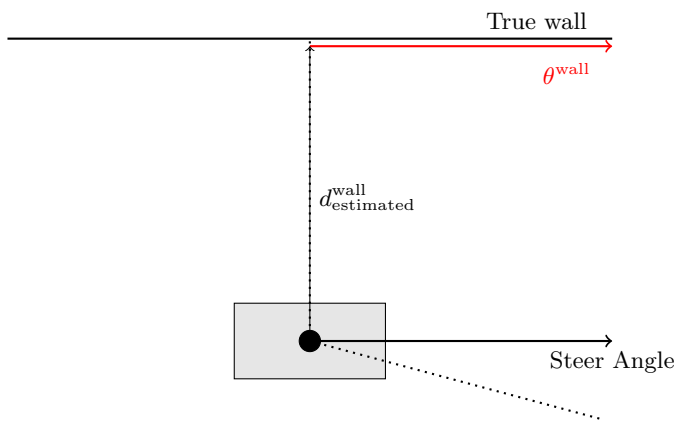

Figure 1: **Wall Estimator Visualization.** The dotted lines show the angular range of lidar points considered for wall estimation. The red arrow represents our estimated wall relative to the true wall. $d^{\text{wall}}_{\text{estimated}}$ is the perpendicular distance from the car to the estimated wall, and θ_{wall} is the wall orientation relative to the car frame.

2. PD Controller

Steering commands are generated using a proportional-derivative (PD) controller that regulates the lateral distance between the vehicle and the estimated wall. Let $s \in \{-1, +1\}$ denote the tracking side ($+1$ for left, -1 for right), d^* the desired wall distance, and $d^{\text{wall}}_{\text{estimated}}$ the estimated distance. The signed distance error is defined as

$$e = -sd^* - d^{\text{wall}}_{\text{estimated}} \tag{4}$$

The derivative term is computed using the controller update period Δt:

$$\dot{e} = \frac{e - e_{\text{prev}}}{\Delta t}, \qquad \Delta t = 0.02 \text{ s} \tag{5}$$

The final steering command combines distance regulation and wall alignment:

$$\delta = K_p e + K_d \dot{e} + K_w \theta_{\text{wall}} \tag{6}$$

where K_p, K_d, and K_w are controller gains. From tuning in simulation, we discovered the gains to be around 2, 0.2, and 2 respectively.

To respect the vehicle's physical limits, the command is saturated to the maximum steering angle $\delta_{\max}$:

$$\delta_{\text{cmd}} = \text{clip}(\delta, -\delta_{\max}, \delta_{\max}) \tag{7}$$

This control structure maintains a fixed offset from the wall while encouraging the vehicle to remain parallel to it.

Safety Controller

Author: Vandell Vatel

Our safety controller works by "catching" driving commands that are being published, and stopping the race-car if a command would cause it to crash within a given time-frame. It works by projecting a path that our race-car would follow, and ensuring that any obstacles detected by our LiDAR are far enough away from the path to not be hit, creating a "safety radius" around our car.

1. Path Projection

The path projection computes a line that the race-car will follow based on the velocity extracted from the intercepted drive command. For now, we have simplified the projected path to be forward, in the x-direction, from the frame of the race-car. Since we run the line projection every time a drive command is published, at a rate of 50 Hz, we assume slight deviations in the race-car's angle with respect to the line will be caught by future iterations. The computed line is therefore a straight line forward from the race-car that spans a projected distance d_{proj} as a function of Δt, the time frame of the projection, v, the velocity of the race-car, and r_{safe}, the safety radius of our car:

$$d_{proj} = r_{const} * v + r_{safe} \tag{8}$$

$$\vec{v}_{path} = [d_{proj}, 0, 0] \tag{9}$$

The values of Δt and r_{safe} were tuned through testing detailed later in Experimental Evaluation.

2. Crash Detection

Taking this projected path, we check if there are any obstacles that our race-car may crash into it, and if so, issue a command overriding the intercepted drive command, telling our race-car to stop. The most recent LiDAR data is filtered to only include points that are within angle ranges [-$pi/4$, $pi/4$] and that are a distance d_{proj} away from the race-car shown in Figure 2 as a cone in front of the car. We assume that points filtered out by this are too far from the path to ever realistically be collided with. For each of the filtered points, we calculate the cross product of the point p and the normalized path vector to find the distance of the point from the projected path.

$$\delta_p = p \times \frac{\vec{v}_{path}}{|\vec{v}_{path}|} \tag{10}$$

$$\exists p : \delta_p < r_{safe} \tag{11}$$

If the δ_p of any of these points is less than r_{safe}, it would mean that there are obstacles in the current projected path of the race-car, that are not far away enough to not be crashed into. This is shown in the left scenario of Figure 2, where the red circle represents r_{safe} and the red arrow represents a $\delta_p < r_{safe}$.

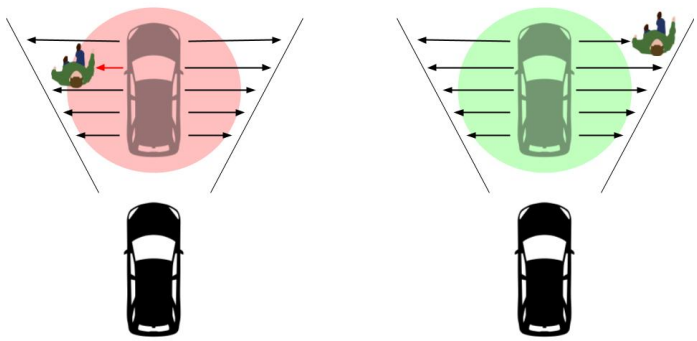

Figure 2: **Crash Detection Visualization.** (*Left*) A bystander is detected within the safety radius of the race-car. (*Right*) No obstacles are found and the race-car is free to proceed.

3 Experimental Evaluation

3.1 Wall Follower Evaluation

Author: Nina Ruiz-Garcia

We evaluated the wall follower using two metrics. The first is the estimated error defined in Equation (4). A positive error indicates the robot is too far from the wall, while a negative error indicates it's too close. The second metric measures the accuracy of this estimate by comparing the estimated error e with the measured deviation from the desired distance, e_{observed}. The difference between these values is defined as

$$d_{error} = e - e_{\text{observed}}. \tag{12}$$

Lower magnitudes of both metrics indicate better performance.

Initially, the wall follower was tested at 1 m/s with a desired distance of 0.6 m on a concrete surface. The robot exhibited consistent behavior across runs, which is reflected in the error plot shown in Figure 3.

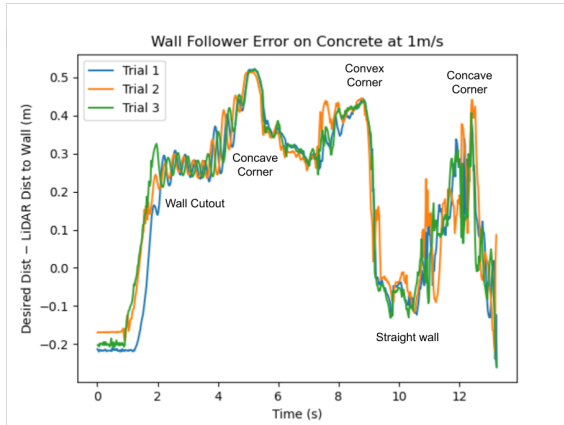

Figure 3: **Error plot from the initial wall follower test.** The wall cutout corresponds to a recessed doorway. Concave corners occur when the robot turns along the inside of a $\frac{\pi}{4}$ rad corner, while the convex corner represents the outside of a $\frac{5\pi}{4}$ rad corner. Error peaks correspond to these features.

Error peaks at both convex and concave corners indicate the controller does not turn tightly enough when encountering sharp wall features. Tuning the proportional gain K_p would likely reduce the magnitude of error.

Next, we evaluated the robot with a different starting angles along the same wall. We initialized the robot at 1 m/s with a 0.6 m desired distance and $\frac{\pi}{4}$ rad angle toward the wall. An outward-facing orientation was not tested because the robot performed well on an analogous wall feature (a convex corner followed by a straight wall).

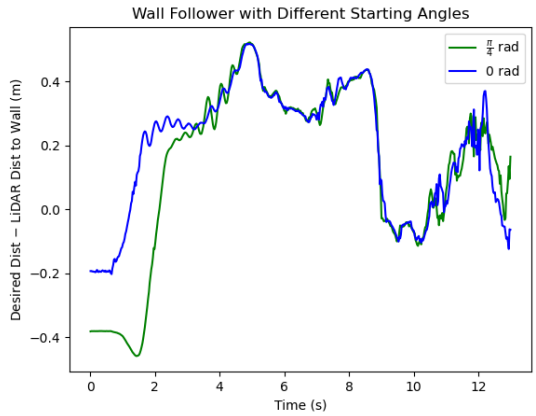

Figure 4: **Average error across three trials for different starting angles.** Both starting configurations converge. The angled start briefly increases error before converging after about three seconds and shows slightly larger oscillations between 3 and 4.5 seconds.

To further investigate oscillatory behavior, we tested the controller along a long straight wall leading to a pronounced cutout. The robot began parallel to the wall (0 rad orientation) with a 0.6 m desired distance on concrete flooring. This setup isolated the controller response along the straight wall and highlighted oscillation amplification caused by the cutout.

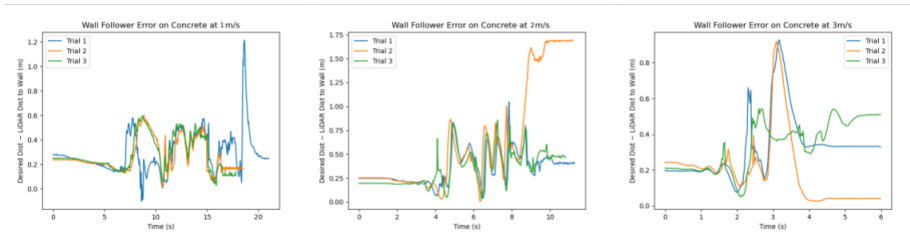

Figure 5: **Error plots for oscillation experiments at different speeds.** Oscillatory behavior appears in all trials and speeds.

Persistent oscillations across all speeds suggest insufficient damping in the controller. Increasing the derivative gain K_d should reduce oscillation amplitude and improve stability.

Finally, we evaluated the accuracy of the estimated wall distance. The robot followed a straight wall on carpet at 1 m/s with a 0.6 m desired distance. The

desired distance was marked on the floor, and observations showed a consistent positive error, which is also reflected in Figure 6. Table 1 shows the steady-state error is consistent across trials.

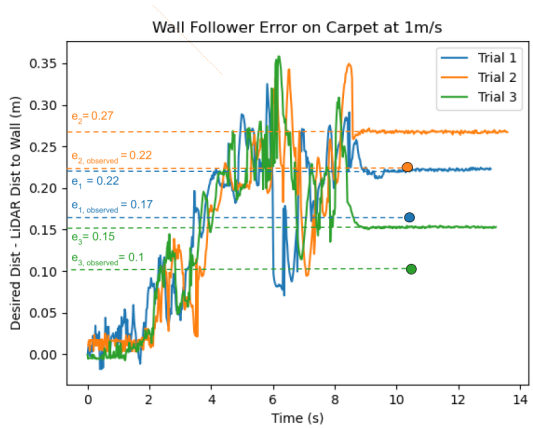

Figure 6: **Error plot for $d_{\mathbf{error}}$ accuracy experiment.** Estimated errors are consistently larger than observed measurements.

Table 1: Accuracy of error measurement.

	$d_{estimated}$	$d_{observed}$	d_{error}
Trial 1	0.22	0.17	0.05
Trial 2	0.27	0.22	0.05
Trial 3	0.15	0.10	0.05

Because the estimated error consistently exceeds the observed value, the controller maintains a small steady-state offset. Adding an integral term could help eliminate this bias by compensating for persistent error.

3.2 Safety Controller Evaluation

Author: Brendon Murphy

We evaluated the safety controller using two metrics: command distance, $d_{\text{cmd}}^{\text{safety}}$, and the safety estimation error, defined as:

$$e^{\text{safety}} = |d_{\text{estimated}}^{\text{safety}} - d_{\text{observed}}^{\text{safety}}|$$

Here, $d_{\text{estimated}}^{\text{safety}}$ is the simulated distance to the wall, $d_{\text{observed}}^{\text{safety}}$ is the physical measurement, and $d_{\text{cmd}}^{\text{safety}}$ is the distance triggering the zero-velocity safety command.

Because varying surface and velocity alters car behavior, establishing a single target distance $d^{\text{wall}}_{\text{desired}}$ was difficult. Thus, we qualitatively evaluated our results, carefully tuning parameters r_{const} and r_{safe} in Equation (8) until we achieved a satisfactory baseline for each trial.

We conducted three trials on carpet and concrete surfaces at velocities of 1, 2, and 3. We utilized concrete trials as our primary baseline because its lower friction makes stopping harder. Figure 7 shows our tuned parameters yielded a non-linear relationship between stopping distances and velocities, reflecting deceleration physics. Additionally, $d^{\text{safety}}_{\text{estimated}}$ and $d^{\text{safety}}_{\text{observed}}$ matched consistently, proving the robot accurately tracks its physical location in the world. We also observed $d^{\text{safety}}_{\text{cmd}}$ increasing monotonically with velocity, matching expectations from altering v in Equation (8).

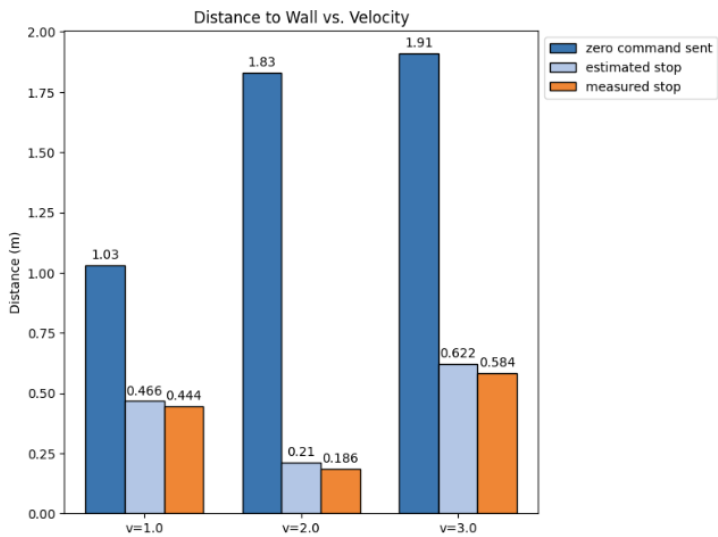

Figure 7: **Stopping distances from the wall.** Each group of bars represents the average of three trials at the specified velocity. The bars represent d_{cmd}, $d_{\text{estimated}}$, and d_{observed}, respectively.

	$x = 2$	$x = 1.5$	$x = 1$
$v = 1$	good	good	good
$v = 2$	too close!	too close!	okay
$v = 3$	too far!	too far!	okay

Table 2: **Testing non-linear relationships.** Qualitative assessment varying the v^x term in d'_{proj}.

4 Conclusion

Author: Irene Tang

We successfully implemented a wall follower and a safety controller, identifying key performance characteristics and areas for optimization.

The wall follower demonstrated robust performance in diverse scenarios, consistently navigating convex and concave corners and noisy environments with protruding obstacles, suggesting that the follower generalizes well to real-world variability. Additionally, the follower was robust against different starting angles, meaning that minor changes in initialization did not affect overall performance. However, detailed analysis revealed a systemic limitation: the PD controller had issues with handling oscillations and persistent errors, indicating a steady state bias.

The oscillations stem from the fact that the PD controller cannot correct for sustained directional errors. One way to address this is by adding an integral term for a PID controller to eliminate persistent offsets. Additionally, if we had more time, we would experiment with more stable wall follower approaches, like pure pursuit with RANSAC-enabled spline-fitting.

The safety controller reliably stopped the car before head-on collisions. The stopping distance of the safety controller depended heavily on the speed of the car, which interestingly followed a non-monotonic relationship (opposite to kinematic expectations). To account for this, we will tune the safety controller for each speed range (slow, medium, fast) since there doesn't seem to be one set of parameters that produces consistent results.

5 Lessons Learned

Nina

I learned about the importance of data collection for the purpose of being able to communicate about our work. We started with the ways in which we wanted to evaluate our work, then we looked at what data we would need, and from there we talked planned the bag down to the topics we would need. This made testing go a lot smoother and putting together materials for the briefing and report was easier.

Irene

The simulation to real gap is real - it not only effects the performance of the robot when translating from simulation to real life physics, but it also effects the efficiency and ease of testing. Real world testing requires a lot more effort - from designing tests, writing launch files, and accurately collecting information, testing can certainly be a headache.

Vandell

Always check that you're looking at the correct repository so you don't make a pull request to the class repository

Brendon

Ensuring we have a clean and organized repository is important when collecting and analyzing data. There were many instances where time was wasted trying to find old bag files and metrics that ended up being useful to us. Accurate labeling and naming of files will help us boost efficiency when working together in the long run.